

Diseño e Implementación de una Aplicación Java Basada en MVC y Patrón de Eventos para la visualización de la notación $O(n)$

Vicent Roig Juan
Universitat de les Illes Balears

Resumen—En esta memoria se documenta la Práctica 1 de la asignatura Algoritmia avanzada, cuyo objetivo principal es el desarrollo de una aplicación en Java para representar la diferencia entre distintos costes computacionales, esta debía estructurarse haciendo uso del patrón Modelo Vista Controlador (MVC) y con una arquitectura basada en eventos. La aplicación permite ejecutar algoritmos de complejidades $O(n)$, $O(n \log n)$, $O(n^2)$ y $O(n^3)$, y visualizar gráficamente los tiempos de ejecución.

El sistema se implementa principalmente mediante el siguiente conjunto de clases: `Modelo`, `Vista`, `Controlador`, `Matriz`, así como una clase para cada algoritmo, `Practica_1` que implementa un método `main`, y la interfaz `Algoritmo` con la cual se unifican los distintos algoritmos que representan los diferentes costes computacionales. La lógica de interacción se activa a través de eventos de interfaz gráfica (`ActionListener`) y actualizaciones en el hilo de eventos de Swing.

Como conclusión de la práctica, se ha obtenido una aplicación organizada según el patrón de diseño modelo vista controlador y que, a su vez, hace uso del patrón por eventos para visualizar gráficamente las diferencias entre los distintos costes computacionales.

Index Terms—Java, MVC, patrón de eventos, complejidad temporal, análisis asintótico, Swing, algoritmos.

I. INTRODUCCIÓN

El denominar un software como robusto no puede acotarse únicamente a que este sea funcional, sino que también debe haber sido estructurado para permitir modificación, depuración y reutilización. En esta práctica, se ha combinado tanto el diseño de una interfaz gráfica con interacción como el estudio de la complejidad algorítmica.

La arquitectura que hemos utilizado es el patrón Modelo Vista Controlador (MVC), que es de gran utilidad para poder aislar y organizar los datos y la interacción entre los mismos.

Respecto a los algoritmos, se han implementado y comparado algoritmos con órdenes de complejidad diferentes: búsqueda lineal ($O(n)$), *merge sort* ($O(n \log n)$), *bubble sort* ($O(n^2)$) y multiplicación de matrices ($O(n^3)$).

El objetivo de la práctica es demostrar y consolidar nuestros conocimientos en los patrones de diseño MVC, el patrón de eventos y la complejidad asintótica.

II. DATOS GENERALES DE LA PRÁCTICA

II-A. Entorno de programación

El entorno utilizado para el desarrollo es Java, haciendo uso de Swing para la interfaz visual. El proyecto se organiza mediante clases.

Java es un lenguaje con el cual ya estamos experimentados, con un fuerte componente de orientación a objetos, ideal para el patrón MVC, herramientas para la creación de interfaces gráficas, así como interacciones con las mismas y medición temporal mediante `System.nanoTime()`.

II-B. Objetivos

Los objetivos definidos para la práctica son:

- Aplicar el patrón MVC en una aplicación de Java real.
- Implementar una arquitectura basada en eventos para la interacción de usuario.
- Integrar varios algoritmos representativos de diferentes complejidades.
- Estimar y justificar el coste asintótico a partir de los datos.

III. UTILIDAD DEL MODELO VISTA CONTROLADOR

III-A. Visión general

MVC separa la aplicación en tres capas lógicas:

- **Modelo:** gestiona datos y operaciones sobre estructuras internas.
- **Vista:** representa la información al usuario y captura interacciones.
- **Controlador:** coordina el flujo y hace de intermediario entre modelo y vista.

En la práctica, esta separación evita que la lógica algorítmica se mezcle con código gráfico, estructurando y conteniendo el código para facilitar el mantenimiento.

III-B. Aplicación de MVC en el código

El componente `Modelo` y `Matriz` encapsulan los datos, en este caso, tenemos por un lado arrays aleatorios en `Modelo` para los algoritmos de coste $O(n)$, $O(n \log n)$ y $O(n^2)$ y por otro, las matrices aleatorias de `Matriz` para $O(n^3)$. Con métodos para obtener y cambiar los datos (getters y setters) así como para obtener el tamaño. Por su parte, `Controlador` es el que se encarga de obtener los datos de modelo y pasarlos a vista, coordinando las dos partes.

La `Vista` contiene la interfaz Swing, botones de selección y panel de dibujo para las gráficas. Este módulo no implementa algoritmos directamente; solicita resultados al controlador y trabaja con la información recibida.

Finalmente, el `main Practica_1` inicializa controlador y vista iniciando la interfaz gráfica.

III-C. Ventajas observadas del uso de MVC

1. **Modularidad:** cada clase se centra y encapsula su función.
2. **Escalabilidad:** Añadir nuevos algoritmos no requiere cambiar grandes partes de código o rediseñar la relación entre las clases.
3. **Mantenibilidad:** Se pueden refactorizar las distintas clases de forma independiente sin afectar al resto.

IV. PATRÓN DE EVENTOS

IV-A. Concepto

El patrón de eventos desacopla la ocurrencia de una acción de su tratamiento. La vista emite eventos (clics), y manejadores registrados ejecutan respuestas específicas sin necesidad de acoplar directamente toda la lógica.

En la aplicación, los botones de la interfaz registran `ActionListener` para lanzar el pintado de cada curva de complejidad en hilos separados. Cuando el usuario pulsa uno de los cuatro botones, el `ActionListener` correspondiente invoca `new Thread(...).start()`, delegando la ejecución de la batería de pruebas a un hilo de trabajo distinto del *Event Dispatch Thread* (EDT). De este modo, la interfaz gráfica permanece reactiva durante todo el proceso de medición, que puede prolongarse varios segundos en los algoritmos de mayor coste computacional ($O(n^2)$ y $O(n^3)$). Sin esta separación, el EDT quedaría bloqueado y la ventana dejaría de responder a eventos del sistema operativo, congelándose visualmente.

Listing 1. Registro de `ActionListener` y lanzamiento de hilo en `Vista.java`

```
botones[0].addActionListener(
    e -> new Thread(this::paintLinealGraph).start());
botones[1].addActionListener(
    e -> new Thread(this::paintNlognGraph).start());
botones[2].addActionListener(
    e -> new Thread(this::paintNsquaredGraph).start());
botones[3].addActionListener(
    e -> new Thread(this::paintNcubicGraph).start());
```

Cada referencia a método (`this::paintLinealGraph`, etc.) encapsula la lógica completa de un algoritmo: creación del modelo, instanciación del algoritmo, ejecución de la batería de pruebas y posterior llamada al método de dibujado. El uso de expresiones lambda junto con referencias a método, disponible a partir de Java 8, reduce la verbosidad respecto a la alternativa de clases anónimas y mejora la legibilidad del código.

Además, `MouseListener` se utiliza para efectos visuales de interacción. Concretamente se emplea la subinterfaz `MouseAdapter`, que permite sobrescribir únicamente los métodos `mouseEntered` y `mouseExited` sin necesidad de implementar los cinco métodos de la interfaz completa. Cuando el cursor entra en el área de un botón, su color de fondo cambia a una tonalidad más oscura; al salir, recupera el color original. Este comportamiento de *hover* refuerza la retroalimentación visual al usuario, indicando qué elemento es interactuable antes de que se produzca el clic.

Listing 2. Efecto hover mediante `MouseAdapter` en `Vista.java`

```
boton.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseEntered(MouseEvent evt) {
        boton.setBackground(new Color(30, 90, 170)); //
        tono mas oscuro
    }
    @Override
    public void mouseExited(MouseEvent evt) {
        boton.setBackground(new Color(60, 120, 200)); //
        color base
    }
});
```

El patrón queda reforzado con llamadas a `SwingUtilities.invokeLater(...)` para asegurar la actualización segura de los componentes gráficos desde los hilos de trabajo. Swing no es *thread-safe*: todas las modificaciones sobre componentes visuales deben realizarse exclusivamente en el EDT. Invocar `repaint()` o modificar el estado de un `JPanel` directamente desde un hilo secundario puede producir condiciones de carrera que se manifiestan como artefactos visuales, renderizados parciales o, en casos extremos, excepciones no controladas. `SwingUtilities.invokeLater(Runnable)` encola la tarea proporcionada en la cola de eventos del EDT, garantizando que la actualización gráfica se ejecute de forma ordenada una vez concluida la medición:

Listing 3. Uso de `invokeLater` para actualizar la grafica

```
// Desde el hilo de trabajo, tras obtener los datos:
SwingUtilities.invokeLater(() -> panelGrafica.pintarGrafico(
    DIBUJAR_LINEAL));
```

Esta decisión evita incoherencias de renderizado y sigue las recomendaciones oficiales de Oracle para la programación concurrente en aplicaciones Swing. La separación de responsabilidades resulta, pues, doble: por un lado, el hilo de trabajo se ocupa exclusivamente del cómputo intensivo; por otro, el EDT retiene el control absoluto sobre el estado visual de la aplicación.

Para reforzar la exclusión mutua entre los distintos hilos de trabajo —ya que varios botones podrían pulsarse en rápida sucesión—, la clase `Vista` declara un `Semaphore` de capacidad 1. Antes de iniciar el pintado, el hilo adquiere el permiso (`mutex.acquire()`); al terminar, lo libera (`mutex.release()`). Esto garantiza que en ningún momento dos hilos intenten escribir simultáneamente sobre el mismo `Graphics2D`, lo que podría corromper la imagen renderizada.

Listing 4. Semaforo de exclusion mutua en `Vista.java`

```
private Semaphore mutex = new Semaphore(1);

// Dentro del metodo de pintado:
try {
    mutex.acquire();
    // ... obtencion de datos y dibujado ...
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
} finally {
    mutex.release();
}
```

En conjunto, la combinación de `ActionListener` para el despacho de hilos, `MouseAdapter` para

la retroalimentación visual, el uso correcto de `SwingUtilities.invokeLater` y el `Semaphore` de exclusión mutua constituye una implementación sólida del modelo de concurrencia recomendado en Swing, equilibrando rendimiento, seguridad y experiencia de usuario.

IV-B. Resumen de utilidad

La arquitectura orientada a eventos aporta:

- Respuesta inmediata ante acciones del usuario.
- Mayor separación entre la parte de interfaz y de cálculo.
- Mejor organización del flujo de ejecución de la interfaz gráfica.

IV-C. Modelo de clases generado

V. DESCRIPCIÓN DE LA PRÁCTICA DESARROLLADA

V-A. Estructura de clases

El proyecto incluye las siguientes piezas principales:

- Algoritmo: interfaz con método `ejecutarAlgoritmo()`.
- `AlgoritmoLineal`: búsqueda secuencial sobre array.
- `AlgoritmoNLogn`: implementación de *merge sort*.
- `AlgoritmoN2`: implementación de *bubble sort*.
- `AlgoritmoN3`: multiplicación de matrices cuadradas.
- `Matriz`: estructura auxiliar para operaciones cúbicas.
- `Modelo`: generación y gestión de datos de entrada.
- `Controlador`: ejecución repetida y captura de tiempos.
- `Vista`: representación gráfica y botones para seleccionar algoritmo.

Esta distribución por capas y facilita la incorporación de nuevos algoritmos mediante polimorfismo.

V-B. Flujo de ejecución

El flujo funcional se puede resumir como:

1. Inicio de la aplicación y creación de vista/controlador.
2. Usuario selecciona algoritmo desde la interfaz.
3. Controlador crea modelos de tamaño creciente.
4. Se ejecuta el algoritmo y se mide tiempo en nanosegundos.
5. Se repite dicho experimento varias veces y se conserva el mejor tiempo.
6. Dibujado en vista del gráfico de complejidad temporal de cada uno.

V-C. Criterios experimentales adoptados

Para reducir el ruido de medida se realizan múltiples iteraciones por cada tamaño. El uso del mínimo entre repeticiones intenta aproximar el coste “limpio” de los algoritmos, mitigando efectos de carga del sistema, variaciones de planificador y fluctuaciones de memoria que se pudieran producir.

Aunque no elimina todas las fuentes de sesgo, y puede ser algo costoso, se ha optado a esta opción al ser una práctica donde el objetivo es visualizar tendencias de los distintos algoritmos.

VI. CÁLCULO DE COSTE ASINTÓTICO POR INSTRUCCIÓN CRÍTICA

VI-A. Marco de análisis

La instrucción crítica es aquella que domina el tiempo total al aumentar n . En esta práctica, normalmente coincide con comparaciones en bucles o multiplicaciones/sumas dentro de bucles anidados.

El tiempo total se modela de forma simplificada como:

$$T(n) = c \cdot f(n) + \text{términos de orden inferior}, \quad (1)$$

donde $f(n)$ es el número de ejecuciones de la instrucción dominante.

VI-B. Algoritmo lineal $O(n)$

En `buscarElementoEnArray`, la instrucción crítica es la comparación `modelo.getElementoArray(i) == elemento`. En el peor caso no se encuentra el valor y se recorren las n posiciones. Para la práctica, hemos hecho que el algoritmo siempre se encuentra con el peor caso, para representar de mejor manera el coste en el peor de los casos del algoritmo lineal.

$$T_{\text{lineal}}(n) = a \cdot n + b \Rightarrow O(n) \quad (2)$$

VI-C. Algoritmo $O(n \log n)$: *merge sort*

El algoritmo divide recursivamente el problema en dos mitades y combina resultados en tiempo lineal por nivel. La recurrencia estándar es:

$$T(n) = 2T(n/2) + cn, \quad (3)$$

cuya solución es $T(n) = O(n \log n)$.

La instrucción crítica durante la fusión es la comparación entre elementos de subarrays temporales, repetida un número lineal de veces en cada nivel del árbol de recursión.

VI-D. Algoritmo cuadrático $O(n^2)$: *bubble sort*

En `bubbleSort`, la comparación entre posiciones adyacentes se ejecuta de forma anidada. En peor caso, el número de comparaciones es proporcional a $n(n-1)$.

$$T_{\text{bubble}}(n) = a \cdot n^2 + b \cdot n + c \Rightarrow O(n^2) \quad (4)$$

La mejora por bandera `swapped` permite terminar antes en casos parcialmente ordenados, pero no altera el orden asintótico de peor caso.

VI-E. Algoritmo cúbico $O(n^3)$: *producto matricial*

En `Matriz.producto`, la instrucción crítica es la operación `sum += a[i][k] * b[k][j]` dentro de tres bucles anidados. El total de iteraciones es n^3 .

$$T_{\text{matriz}}(n) = a \cdot n^3 + \text{orden inferior} \Rightarrow O(n^3) \quad (5)$$

Este algoritmo resulta útil para mostrar de forma clara cómo pequeñas subidas de n disparan el tiempo de ejecución.

VII. DISCUSIÓN TÉCNICA DE RESULTADOS Y DECISIONES DE DISEÑO

VII-A. Comportamiento esperado

La práctica permite confirmar que el crecimiento temporal depende de la clase de complejidad:

- El algoritmo lineal escala de forma suave.
- $O(n \log n)$ se mantiene eficiente para entradas medias/grandes.
- $O(n^2)$ y $O(n^3)$ crecen rápidamente y limitan tamaño viable.

Esta observación muestra la importancia de seleccionar algoritmos según tamaño de datos y restricciones de rendimiento.

VII-B. Fortalezas de la implementación

Se consideran aciertos principales:

- Interfaz común `Algoritmo`, que facilita polimorfismo.
- Separación clara de vista y lógica de cálculo.
- Integración de visualización de una gráfica para una interpretación intuitiva.

Estos aciertos ayudarán a posibles mejoras de calidad para próximas versiones si se quiere hacer la comparación con otros algoritmos.

VIII. SÍNTESIS: UTILIDAD GLOBAL DE MVC Y DE LOS CONCEPTOS APLICADOS

Esta práctica tiene como principal objetivo integrar tanto la teoría de algoritmos avanzados como el diseño de la interfaz.

MVC permite trabajar de forma ordenada, distribuir tareas entre miembros y reducir conflictos al modificar componentes. El patrón de eventos, por su parte, habilita una interfaz interactiva sin mezclar excesivamente la lógica de dichas componentes.

En conjunto, estos conceptos conforman una base sólida para proyectos más complejos: aplicaciones con más pantallas, nuevos modelos de datos o experimentos de rendimiento de mayor escala.

IX. DISEÑO DETALLADO DE COMPONENTES

IX-A. Contrato común de algoritmos

El uso de la interfaz `Algoritmo` establece un contrato mínimo estable para cualquier estrategia de cálculo. Desde la perspectiva de ingeniería de software, este enfoque presenta dos ventajas clave: primero, estandariza la invocación de algoritmos desde el controlador; segundo, reduce el impacto de cambios locales sobre el resto del sistema. La firma `long ejecutarAlgoritmo()` abstrae internamente tanto el tipo de datos como la estrategia usada.

Listing 5. Interfaz común de ejecución

```
public interface Algoritmo {  
    public long ejecutarAlgoritmo();  
}
```

La utilidad práctica de este contrato se aprecia cuando se añaden implementaciones nuevas: no se reescribe la vista ni se altera la estructura de control general, solo se incorpora una clase concreta que respeta la interfaz.

IX-B. Responsabilidades del controlador

El controlador realiza tareas de coordinación intensivas: define tamaños de entrada, instancia modelos, ejecuta algoritmos y agrega resultados. En la práctica analizada, este módulo contiene también la política experimental (repeticiones y selección de mínimos), por lo que constituye el núcleo operativo del sistema.

Adicionalmente, el controlador desacopla la vista de los detalles del cálculo. La GUI no conoce cómo se construye cada instancia de `Modelo` ni qué algoritmo concreto se lanza en cada caso, lo que reduce la complejidad cognitiva del código de presentación.

IX-C. Modelo de datos y generación de entradas

El `Modelo` encapsula un array de enteros y proporciona operaciones elementales de acceso y mutación. El constructor genera entradas pseudoaleatorias, lo que ofrece variedad de casos de prueba sin necesidad de cargar ficheros externos.

IX-D. Vista y renderizado incremental

La vista implementa una interfaz Swing con panel de dibujo persistente mediante `BufferedImage`. Esta decisión permite repintado eficiente y acumulación de curvas sin recalcular datos ya representados. El diseño incorpora además una leyenda lateral, ejes cartesianos y etiquetas semánticas para facilitar la lectura por parte del usuario.

Desde un enfoque de experiencia de usuario, la interfaz presenta una relación directa entre acción y resultado: cada botón activa un experimento y genera su curva correspondiente, reforzando el aprendizaje visual de la complejidad algorítmica.

X. METODOLOGÍA DE VALIDACIÓN Y ANÁLISIS

X-A. Estrategia de medidas

La medición se basa en `System.nanoTime()`, adecuado para intervalos cortos y comparaciones relativas. Para cada tamaño de entrada se repite el experimento varias veces y se conserva el valor mínimo. Esta estrategia persigue aproximar el coste “base” del algoritmo bajo condiciones favorables de ejecución.

No obstante, es importante reconocer que la ejecución en JVM introduce efectos adicionales: calentamiento del compilador JIT, presión del recolector de basura, variaciones de planificación del sistema operativo y competencia por recursos de CPU. Por ello, el resultado debe interpretarse como evidencia orientativa y no como benchmark absoluto.

X-B. Tamaños de entrada y equilibrio experimental

Los tamaños empleados no son idénticos para todas las complejidades. Esta decisión es correcta desde un punto de vista práctico: ejecutar $O(n^3)$ con tamaños muy altos resultaría prohibitivo, mientras que para $O(n)$ sería insuficiente para observar tendencias claras. Ajustar rangos por familia de complejidad permite mantener tiempos razonables sin perder valor comparativo.

Cuadro I
CONFIGURACIÓN GENERAL DE EXPERIMENTOS

Complejidad	Repeticiones	Tamaño máximo aprox.	Estructura
$O(n)$	50	alto	Array
$O(n \log n)$	25	medio-alto	Array
$O(n^2)$	25	medio	Array
$O(n^3)$	4	bajo	Matriz

X-C. Criterios de calidad del software

Además del rendimiento, se han considerado criterios de calidad relevantes para la asignatura:

- **Legibilidad:** nombres de clases alineados con su función.
- **Acoplamiento:** interfaz de algoritmo para minimizar dependencias.
- **Cohesión:** responsabilidades concentradas por módulo.
- **Mantenibilidad:** posibilidad de añadir funcionalidades sin reestructuración global.

Estos criterios refuerzan que la práctica no se limita a “medir tiempos”, sino que integra diseño de software y análisis computacional.

XI. CONCLUSIONES

La Práctica 1 cumple con los objetivos académicos planteados. Se ha construido una aplicación Java funcional con arquitectura MVC y gestión de interacción basada en eventos, y se han estudiado diferentes órdenes de complejidad mediante mediciones experimentales.

Los resultados conceptuales son consistentes con la teoría: los algoritmos de orden polinómico alto presentan peor escalabilidad, mientras que enfoques lineales o $n \log n$ ofrecen mejor comportamiento para entradas crecientes. Además, la separación de responsabilidades proporciona una base mantenible y extensible.

XII. ANÁLISIS COMPARATIVO DE COMPLEJIDADES EN CONTEXTO DOCENTE

Con independencia de los valores numéricos concretos obtenidos en una ejecución, el principal valor pedagógico de la práctica reside en observar la forma de crecimiento.

Cuando el tamaño de entrada crece, las diferencias entre $O(n)$, $O(n \log n)$, $O(n^2)$ y $O(n^3)$ dejan de ser teóricas y se transforman en tiempos de espera visibles para el usuario. Este efecto tiene implicaciones directas en diseño de software real: seleccionar una estrategia adecuada puede evitar cuellos de botella, reducir consumo energético y mejorar experiencia de uso.

Además, comparar familias de algoritmos en un mismo entorno de ejecución (misma JVM, misma máquina, mismo marco de medición) ayuda a eliminar sesgos de interpretación. Aunque no sea un benchmark industrial, sí proporciona una base sólida para justificar decisiones técnicas en informes académicos.

Este enfoque integral resulta especialmente útil en etapas tempranas de formación, donde aún se están consolidando hábitos de ingeniería y documentación profesional.

XIII. ANEXOS TÉCNICOS DE IMPLEMENTACIÓN

En esta sección se incluyen fragmentos representativos de la práctica para reflejar de forma precisa la relación entre arquitectura, eventos y algoritmos. Los extractos se presentan con finalidad documental.

XIII-A. Anexo A: búsqueda lineal

Listing 6. Implementación simplificada de búsqueda lineal

```
public class AlgoritmoLineal implements Algoritmo {
    private Controlador ctrl;
    private Modelo modelo;

    public AlgoritmoLineal(Controlador controlador) {
        this.ctrl = controlador;
        this.modelo = controlador.getModeloLineal();
    }

    @Override
    public long ejecutarAlgoritmo() {
        long tiempoInicial = System.nanoTime();
        buscarElementoEnArray(1000000);
        long tiempoFinal = System.nanoTime();
        return tiempoFinal - tiempoInicial;
    }

    public int buscarElementoEnArray(int elemento) {
        for (int i = 0; i < modelo.getLength(); i++) {
            if (modelo.getElementoArray(i) == elemento) {
                return i;
            }
        }
        return -1;
    }
}
```

XIII-B. Anexo B: ordenación por mezcla

Listing 7. Fragmento de merge sort

```
public class AlgoritmoNLogn implements Algoritmo {
    Controlador cntrl;
    Modelo modelo;

    public AlgoritmoNLogn(Controlador controlador) {
        this.cntrl = controlador;
        this.modelo = cntrl.getModeloNLogn();
    }

    @Override
    public long ejecutarAlgoritmo() {
        long tiempoInicial = System.nanoTime();
        mergeSort(0, modelo.getLength() - 1);
        long tiempoFinal = System.nanoTime();
        return tiempoFinal - tiempoInicial;
    }

    public void mergeSort(int l, int r) {
        if (l < r) {
            int m = l + (r - l) / 2;
            mergeSort(l, m);
            mergeSort(m + 1, r);
            merge(l, m, r);
        }
    }
}
```

XIII-C. Anexo C: producto matricial clásico

Listing 8. Núcleo de multiplicación de matrices

```
public static Matriz producto(Matriz a, Matriz b) {
    Matriz resultado = new Matriz(a.n);
    for (int i = 0; i < a.n; i++) {
        for (int j = 0; j < a.n; j++) {
            int sum = 0;
            for (int k = 0; k < a.n; k++) {
```

```

        sum += a.matriz[i][k] * b.matriz[k][j];
    }
    resultado.matriz[i][j] = sum;
}
return resultado;
}

```

XIII-D. Anexo D: eventos en la vista

Listing 9. Registro de eventos en botones de interfaz

```

botones[0].addActionListener(e -> new Thread(this::
    paintLinealGraph).start());
botones[1].addActionListener(e -> new Thread(this::
    paintNlognGraph).start());
botones[2].addActionListener(e -> new Thread(this::
    paintNsquaredGraph).start());
botones[3].addActionListener(e -> new Thread(this::
    paintNcubicGraph).start());

final JButton boton = botones[i];
boton.addMouseListener(new MouseAdapter() {
    public void mouseEntered(MouseEvent evt) {
        boton.setBackground(new Color(30, 90, 170));
    }

    public void mouseExited(MouseEvent evt) {
        boton.setBackground(new Color(60, 120, 200));
    }
});

```

XIII-E. Anexo E: Visualización de resultados

A continuación, se muestra la gráfica con los resultados finales:

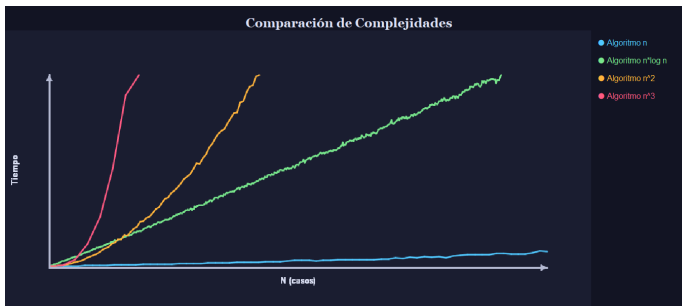


Figura 1. La siguiente imagen pone de manifiesto el distinto orden de complejidad lineal de los algoritmos anteriormente mencionados.

XIII-F. Anexo F: Visualización de resultados

A continuación, se muestra el modelo de clases generado. Los anexos anteriores muestran la coherencia entre la explicación teórica y la implementación real. La interfaz común, la recursividad de merge sort, la triple anidación del producto matricial y la gestión de listeners confirman que la práctica integra exactamente los conceptos solicitados en la asignatura.

XIV. EVALUACIÓN CRÍTICA DEL PROCESO DE DESARROLLO

XIV-A. Riesgos técnicos encontrados

Durante el desarrollo pueden aparecer riesgos habituales:

- **Riesgo de acoplamiento:** incluir lógica algorítmica dentro de la vista.

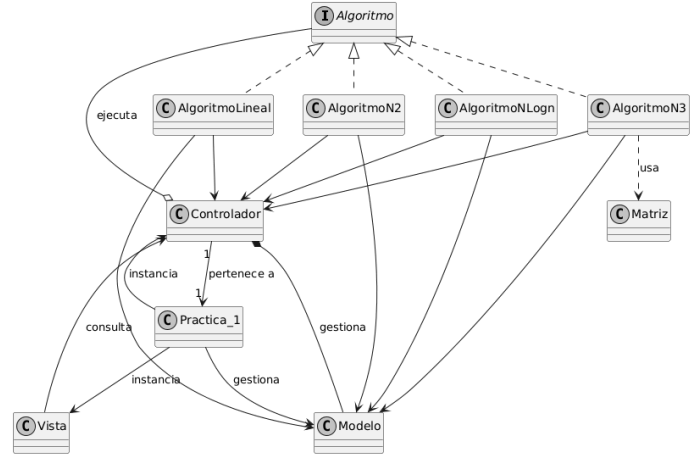


Figura 2. La siguiente imagen pone de manifiesto la relación y organización entre las clases al seguir el patrón de diseño Modelo Vista Controlador.

- **Riesgo de sesgo experimental:** comparar algoritmos con rangos no equivalentes sin justificarlo.
- **Riesgo de concurrencia:** actualizar GUI fuera del hilo de eventos de Swing.
- **Riesgo de interpretación:** confundir tiempos observados con demostración formal.

La mitigación aplicada se basa en separación por capas, criterios explícitos de medida y documentación de supuestos en la memoria.

XIV-B. Lecciones de ingeniería aplicables a futuras prácticas

Las lecciones aprendidas pueden transferirse a otros contextos académicos y profesionales:

1. Definir interfaces desde el inicio facilita evolución incremental.
2. Diseñar la arquitectura antes de codificar reduce deuda técnica.
3. Medir sin contexto puede inducir conclusiones incorrectas.
4. Documentar hipótesis y límites mejora la calidad del informe.
5. Priorizar legibilidad acelera mantenimiento y evaluación docente.

XIV-C. Trabajo futuro y posibles extensiones

Como continuación natural de esta práctica, una primera línea de mejora sería reforzar el bloque experimental con métricas estadísticas adicionales. En lugar de conservar únicamente el mínimo tiempo por tamaño de entrada, resultaría útil almacenar también media, mediana y desviación estándar. Esto permitiría interpretar con mayor precisión la estabilidad de cada algoritmo y distinguir entre mejoras reales y variaciones puntuales de ejecución.

Otra mejora relevante consiste en diversificar los conjuntos de prueba. Actualmente, el enfoque se centra en entradas generadas aleatoriamente, lo cual es adecuado para una primera

aproximación. Sin embargo, incluir casos ordenados, inversamente ordenados y casi ordenados aportaría una visión más completa del comportamiento de cada técnica, especialmente en algoritmos sensibles al estado inicial de los datos.

Desde el punto de vista de visualización, la aplicación puede evolucionar hacia una comparación más didáctica entre curvas teóricas y medidas empíricas. Superponer en la misma gráfica una curva normalizada ideal para $O(n)$, $O(n \log n)$, $O(n^2)$ y $O(n^3)$ ayudaría a interpretar de manera inmediata la tendencia real obtenida y su proximidad respecto al modelo matemático esperado.

En el plano de arquitectura, el diseño MVC actual facilita ampliar funcionalidades sin romper la base existente. Por ejemplo, se podría añadir una capa de persistencia para guardar resultados de experimentos, fechas de ejecución y parámetros utilizados. Esto abriría la puerta a comparar sesiones distintas y elaborar informes evolutivos de manera automática.

También se propone incorporar una batería de pruebas automatizadas para validar tanto la corrección funcional como el comportamiento de componentes clave. Aunque en prácticas académicas esto no siempre se exige de forma estricta, su integración mejora la calidad global del proyecto y reduce el riesgo de introducir regresiones al aplicar cambios sobre algoritmos o sobre la lógica del controlador.

Finalmente, esta práctica puede servir como base para una versión orientada a docencia avanzada, donde el usuario seleccione el algoritmo, configure tamaños de entrada, repeticiones y tipo de distribución, y obtenga un informe final exportable. Esta línea de evolución consolidaría el valor del proyecto no solo como ejercicio de asignatura, sino como herramienta reutilizable para análisis de complejidad en cursos posteriores.

XIV-D. Reflexión final del equipo

La elaboración de la práctica ha permitido comprobar que el desarrollo de software eficiente requiere combinar fundamentos teóricos con buenas decisiones de diseño. El análisis asintótico ofrece una guía sólida para anticipar el rendimiento, pero su aplicación práctica necesita una arquitectura ordenada que facilite experimentar, medir y visualizar resultados con claridad.

Del mismo modo, trabajar con MVC y patrón de eventos en una aplicación completa ha reforzado competencias de organización del código, reparto de responsabilidades y mantenimiento. En conjunto, la experiencia adquirida representa una base técnica útil para afrontar proyectos de mayor complejidad, donde la claridad estructural y la eficiencia algorítmica son factores determinantes.

XIV-E. Cierre de la práctica

Como cierre, esta práctica demuestra que la combinación de patrones de diseño (MVC + eventos) y análisis de complejidad ofrece una aproximación completa al desarrollo de software. No se trata solo de implementar funcionalidades, sino de justificar arquitectónicamente las decisiones y validar empíricamente el comportamiento.

Este enfoque prepara al alumnado para proyectos de mayor escala, en los que la estructura del sistema y la eficiencia de algoritmos determinan el éxito técnico. Por ello, la práctica constituye una base sólida para continuar con asignaturas avanzadas de ingeniería del software, estructuras de datos y optimización.

AGRADECIMIENTOS

Se agradece al profesorado de la asignatura por la guía metodológica, y al equipo de trabajo por la colaboración en análisis, implementación y validación.